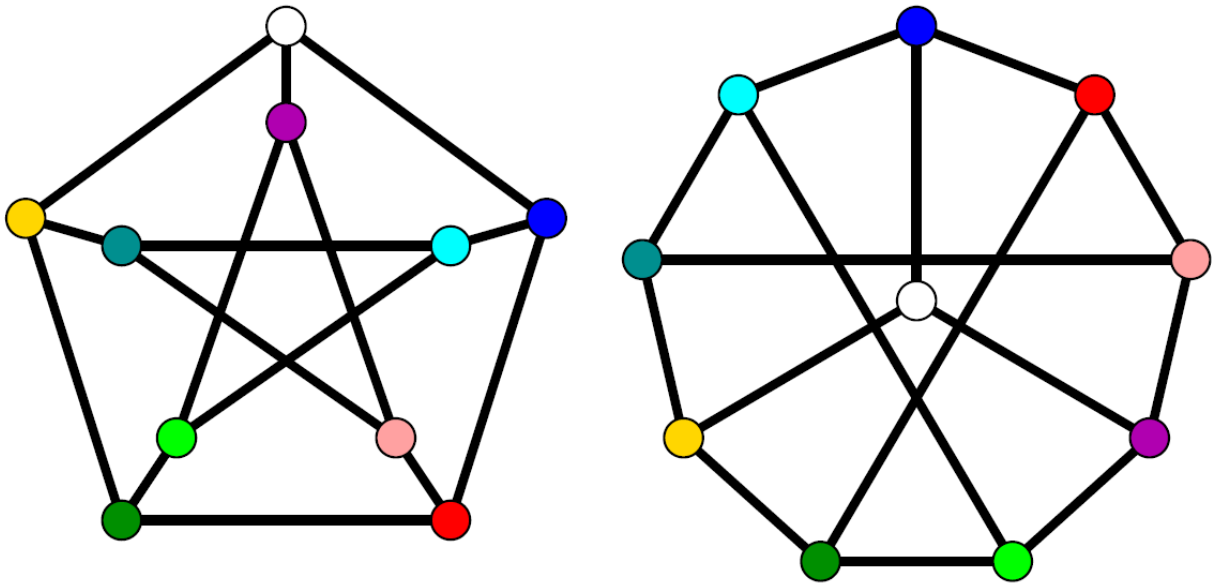
Implementation Project: Graph Isomorphism



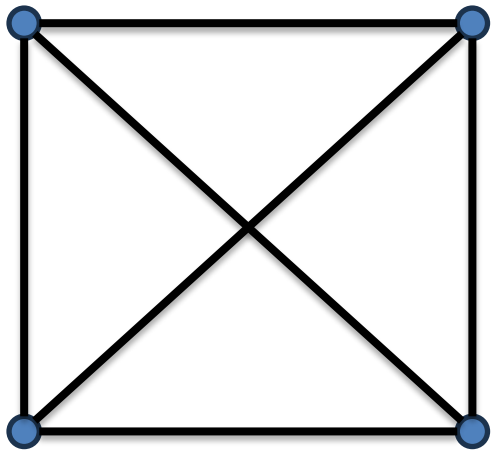
Introduction to a Graph Class for python™

Ruben Hoeksma

Let's talk about graphs

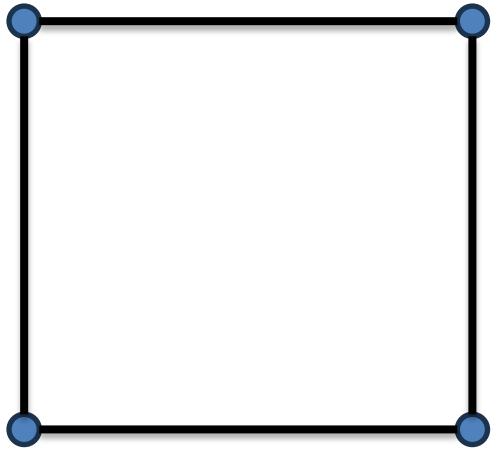
A graph $G(V, E)$ consists of a set of vertices (nodes) V, and a set of edges E.
An edge e is a vertex pair: $e = \{u, v\}$ (undirected), or $e = (u, v)$ (directed), for some $u, v \in V$.

Let’s talk about graphs



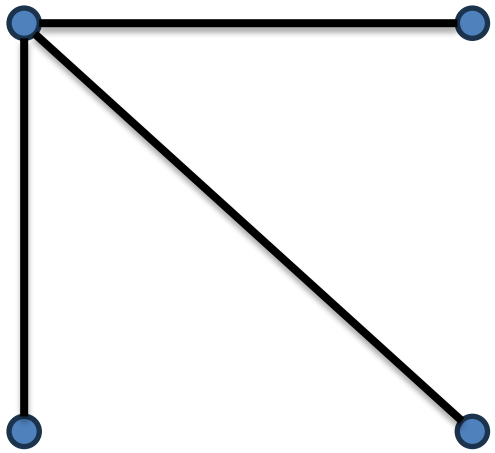
Complete graph

Let’s talk about graphs



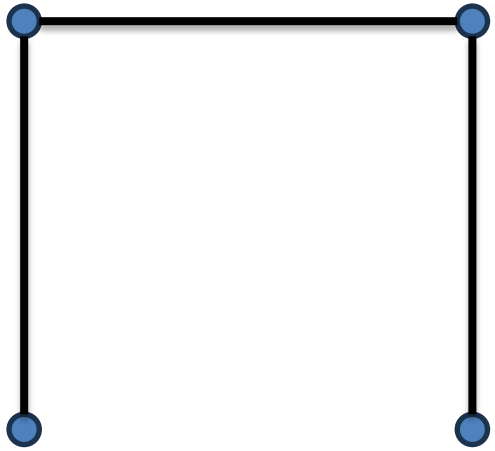
Cycle

Let’s talk about graphs



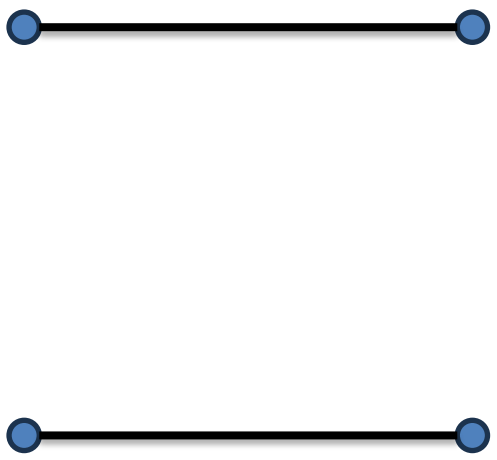
Tree

Let’s talk about graphs



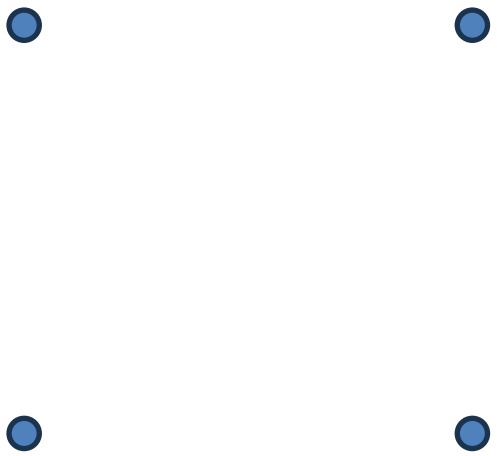
Path (also a tree)

Let’s talk about graphs



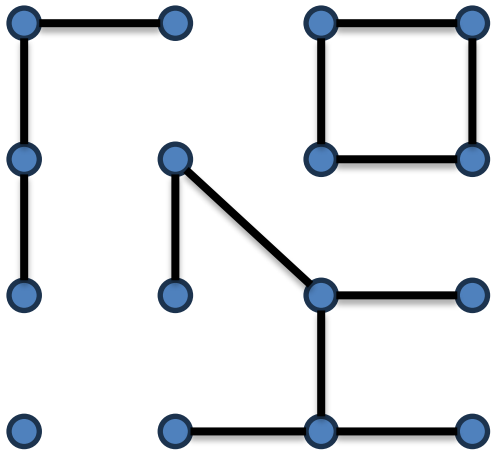
Two paths

Let’s talk about graphs



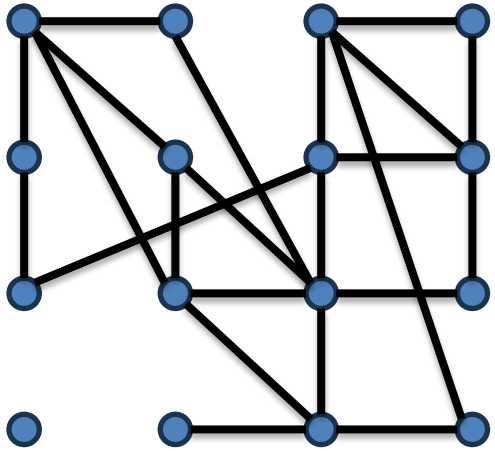
Four vertices

Let’s talk about graphs



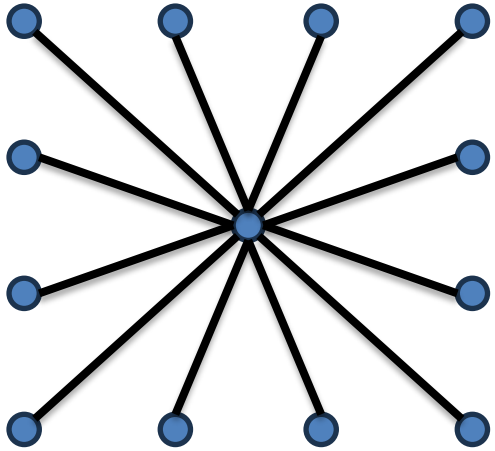
A path, a cycle, a tree, and a vertex

Let’s talk about graphs



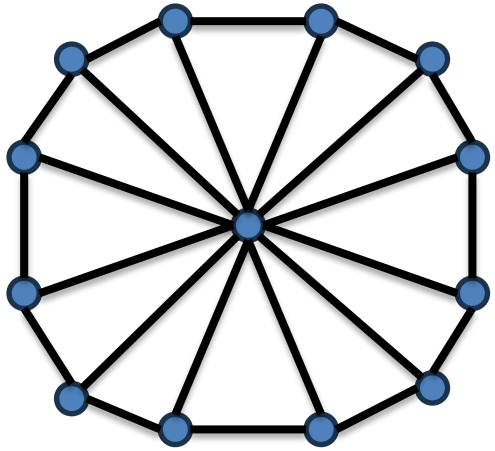
A graph

Let’s talk about graphs



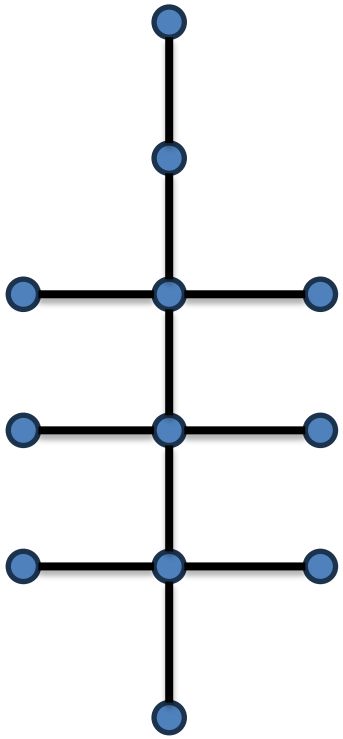
A star (also a tree)

Let’s talk about graphs



A wheel

Let’s talk about graphs



A caterpillar (again, also a tree)

python™

A graph class

Ruben Hoeksma

Not sure what a Python class is?

Check out the videos on Canvas about references, objects and classes in Python.

Project videos - Brief Introduction Into Object Oriented Programming in Python

Back to Project - home

If you are unfamiliar with references, classes and objects in Python or need a reminder, you can watch the videos on this page.

For reference: the slides .

For the first video use:

- recall.py
- append.py
- same.py

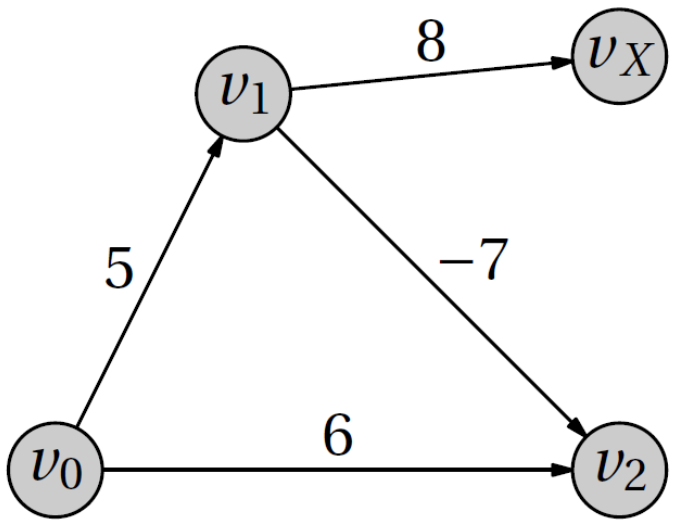
2023-2A

- Home
- Announcements
- OSIRIS Course Information
- Syllabus
- Modules
- Grades
- People
- Export Users
- Assignments
- Collaborations
- BigBlueButton
- Discussions
- Files
- Outcomes
- Pages

For the next video, use linfun.py to follow along. If you want to build a (general) polynomial class, you can use polytest.py .

Graphs

Graph: vertices and (weighted) edges



directed, undirected,
simple, and multi-graphs

File `graph.py` on Canvas:

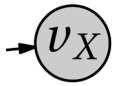
a graph class that you can (should) use for the project

…actually: three classes `Vertex`, `Edge` and `Graph`

Important note: `graph.py` has changed since last year!!!

Class `Vertex`

Attributes:



`_graph` – the `Graph` this vertex belongs to

`label` – the label of this vertex

`_incidence` – a dictionary of $\mathrm{Vertex} \times \mathrm{Vertex}$ pairs of all edges incident with this vertex

Properties:

`graph` – the `Graph` this vertex belongs to

`incidence` – `list` of edges incident w/this vertex

`neighbours` – `list` of neighbouring vertices

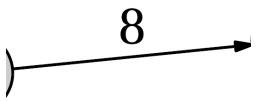
`degree` – degree of the vertex (type `int`)

Methods:

`is_adjacent(other)` – is `other` adjacent to this vertex?

Class Edge

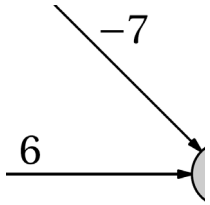
Attributes:



`_tail` – the tail of this edge

`_head` – the head of this edge

`_weight` – the weight of this edge



Properties:

`tail` – the tail `Vertex` of this edge

`head` – the head `Vertex` of this edge

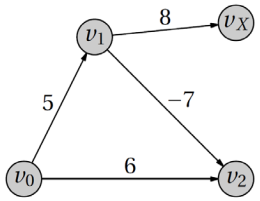
`weight` – the weight of this edge

Methods:

`other_end(one_end)` – `Vertex` at the other end of this edge with `Vertex` `one_end`

`incident(vertex)` – is `vertex` incident with this edge?

Class Graph



Attributes:

`_v` – set of vertices

`_vlist` – list of vertices

`_e` – list of edges

`_simple` – is the graph simple?

`_directed` – is the graph directed?

`_next_label` – number of vertices in graph with default (numbered) labels

Python convention:

Do not use attributes or methods prefixed with _.

They are for internal use only (in `graph.py`).
You only need them if you decide to change `graph.py`.
Otherwise, use the corresponding properties instead…

Class Graph

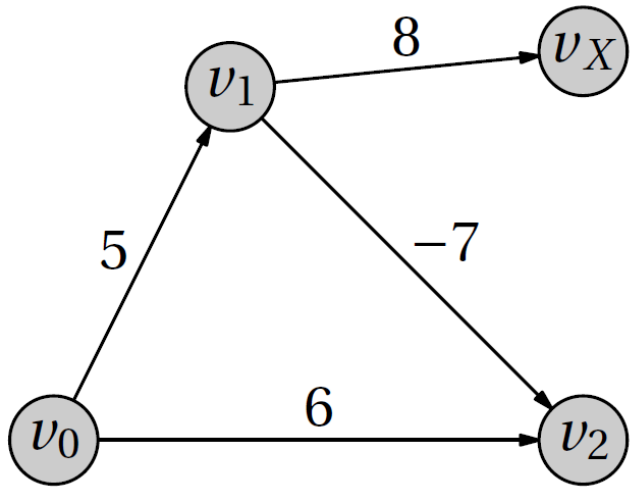
Properties:

`simple` – is this a simple graph?

`directed` – is this a directed graph?

`vertices` – list of all vertices

`edges` – list of all edges



Methods:

`add_vertex(vertex)` – adds `vertex` to this graph

`add_edge(edge)` – adds `edge` to this graph

`find_edge(u, v)` – set of edges between `u` and `v`

`is_adjacent(u, v)` – are vertices `u` and `v` adjacent?

Graph class usage walkthrough

Ruben Hoeksma

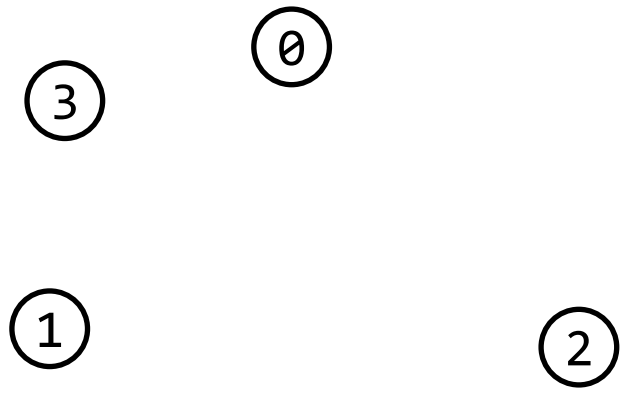
Graph Walkthrough

Let's build a graph!

```
from graph import *
G = Graph(False, 4)
print(G)
print(G.directed)
print(G.simple)
print(G.edges)
print(G.vertices)
```

```
V=[0, 1, 2, 3]
E=[]
False
False
[]
[Vertex(label=0, #incident=0),
Vertex(label=1, #incident=0),
Vertex(label=2, #incident=0),
Vertex(label=3, #incident=0)]
```

also on Canvas
(graph-walkthrough.py # Slide 1**)**



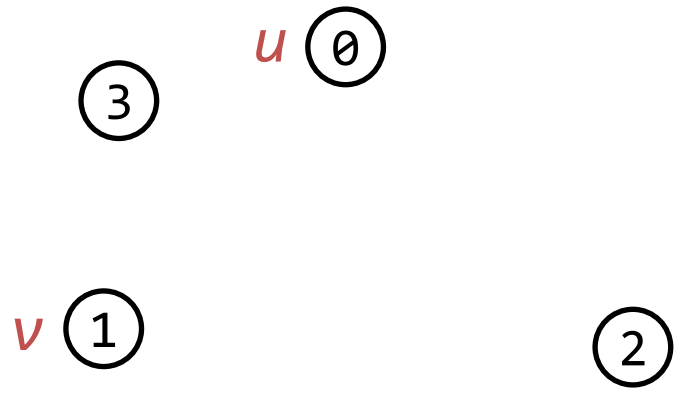
Graph Walkthrough

Pick two vertices:

```
gv = sorted(G.vertices, key=lambda v: v.label)
u = gv[0]
print(type(u))
print(u)
v = gv[1]
print(v)
```

also on Canvas
(graph-walkthrough.py # Slide 2)

```
<class 'graph.Vertex'>
0
1
```



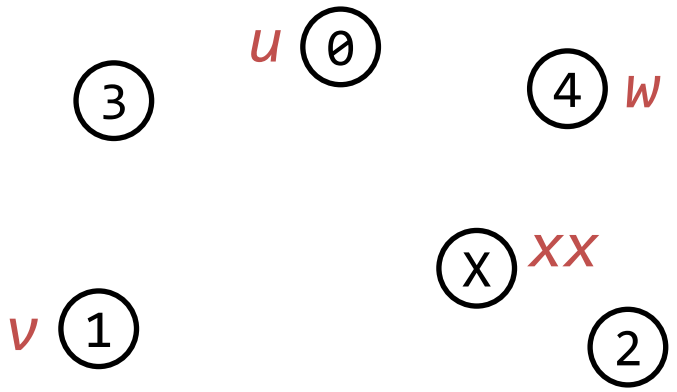
Graph Walkthrough

Our graph needs more vertices.

```
w = Vertex(G)
xx = Vertex(G, 'X')
print(w)
print(xx)
print(G)
print(xx.graph is G)
G.add_vertex(w)
G.add_vertex(xx)
print(G)
```

also on Canvas
(graph-walkthrough.py # Slide 3)

```
4
X
V=[0, 1, 2, 3]
E=[]
True
V=[0, 1, 2, 3, 4, X]
E=[]
```

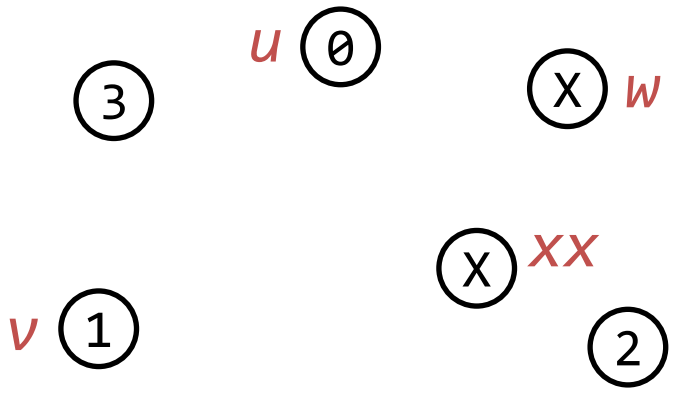


Graph Walkthrough

Our graph needs more vertices.

```
w = Vertex(G, 'X')
xx = Vertex(G, 'X')
print(w)
print(xx)
print(G)
print(xx.graph is G)
G.add_vertex(w)
G.add_vertex(xx)
print(G)
```

```
X
X
V=[0, 1, 2, 3]
E=[]
True
V=[0, 1, 2, 3, X, X]
E=[]
```



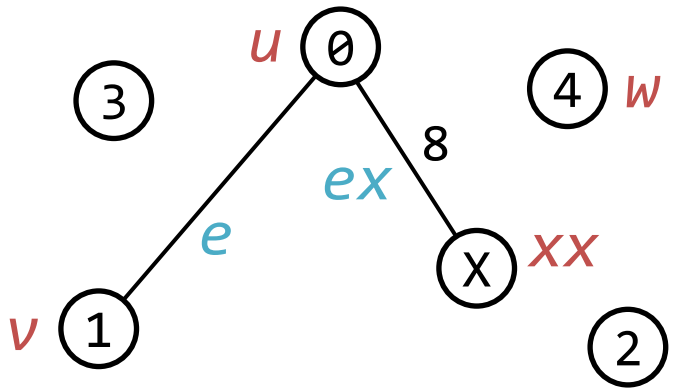
Graph Walkthrough

Our graph also lacks edges.

```
e = Edge(u, v)
ex = Edge(u, xx, 8)
print(e.weight)
print(ex.weight)
print(e)
print(ex)
G.add_edge(e)
G += ex
print(G)
```

also on Canvas
(graph-walkthrough.py # Slide 4**)**

```
None
8
(0, 1)
(0, X)
V=[0, 1, 2, 3, 4, X]
E=[(0, 1), (0, X)]
```



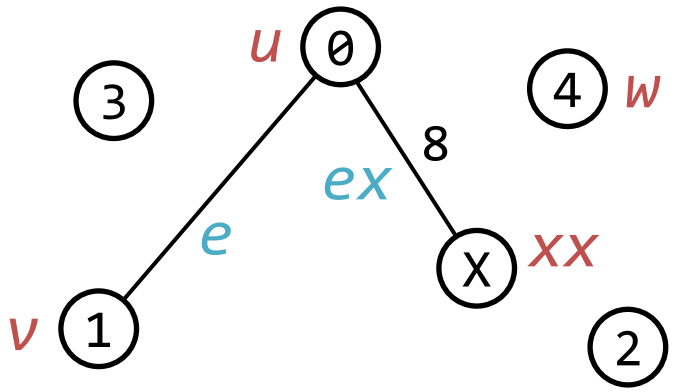
Graph Walkthrough

Now test the graph:

```
print(G.is_adjacent(u, v))
ued = u.incidence
print(type(ued[0]))
print(ued[0])
print(ued[1])
print(u.degree)
```

```
True
<class 'graph.Edge'>
(0, 1)
(0, X)
2
```

also on Canvas
(graph-walkthrough.py # Slide 5)



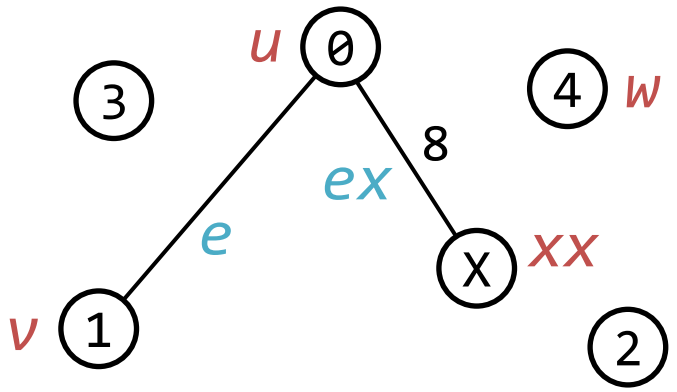
Graph Walkthrough

also test the edges:

```
print(e.tail)
print(e.tail == u)
print(e.head)
print(e.other_end(u))
print(w in G.vertices)
print(e in G.edges)
print(v in u.neighbours)
print(e.incident(w))
```

```
0
True
1
1
True
True
True
False
```

also on Canvas
(graph-walkthrough.py # Slide 6)



During the practical (Thursday)

- Experiment with the Graph Class
  - Add some graph-manipulation algorithms
- Experiment with visualisation of graphs from the graph class using Gephi:
  - Fun puzzles!!
- Implement some algorithms for the Graph Class (shortest path - optional)

Not during the practical but don’t forget:

- Individual assignment Team role inventory test (deadline Friday).